SQL Views — Detailed Notes

1. What is a View?

A **View** in SQL is a **virtual table** created from the result of a `SELECT` query.

A view does **not normally store the actual data separately**. Instead, it stores the SQL query definition. Whenever we access the view, the database uses that query to produce the result.

Simple Definition

> **A View is a virtual table based on the result of a SQL query.**

Think of it like a **saved SELECT query that behaves like a table**.

---

2. Why Do We Need Views?

Suppose we have a large `student` table:

```
CREATE TABLE student (
    sid INT,
    name VARCHAR(50),
    email VARCHAR(100),
    course VARCHAR(50),
    fees INT,
    marks INT
);
```

Now suppose teachers frequently need only:

```
SELECT sid, name, course, marks
FROM student;
```

Instead of writing this query again and again, we can create a view:

```
CREATE VIEW student_details AS
SELECT sid, name, course, marks
FROM student;
```

Now simply:

```
SELECT * FROM student_details;
```

Important Idea

```
Original Table
      ↓
   SELECT
     ↓
    VIEW
     ↓
SELECT from VIEW
```

---

3. Table vs View

Feature	Table	View

Feature	Table	View
Physical storage	Stores data	Usually stores query definition
Data	Actual data	Result derived from tables
Created using	`CREATE TABLE`	`CREATE VIEW`
Can contain rows/columns	Yes	Yes
Can be queried	Yes	Yes
Can have indexes	Yes	Generally no normal indexes
Can simplify complex queries	Not its main purpose	Yes
Can improve security	Limited	Yes
Automatically reflects base-table changes	Data itself changes	Usually yes

Example

Table:

```
student
```

View:

```
student_details
```

The view might show:

```
sid    name       course      marks
-----------------------------------
101    Rahul      Java        85
102    Aman       Python      78
103    Priya      Java        91
```

The actual data remains in the underlying table.

---

4. Syntax of CREATE VIEW

Basic syntax:

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Example

```
CREATE VIEW java_students AS
SELECT sid, name, marks
FROM student
WHERE course = 'Java';
```

Now:

```
SELECT * FROM java_students;
```

Output:

```
sid    name       marks
----------------------
101    Rahul      85
103    Priya      91
```

---

5. Creating a View from Multiple Columns

```
CREATE VIEW student_basic_details AS
SELECT sid, name, email, course
FROM student;
```

Use:

```
SELECT *
FROM student_basic_details;
```

You can also select particular columns:

```
SELECT name, course
FROM student_basic_details;
```

---

6. Creating a View with WHERE

Views can contain conditions.

```
CREATE VIEW passed_students AS
SELECT sid, name, marks
FROM student
WHERE marks >= 40;
```

Query:

```
SELECT *
FROM passed_students;
```

Only students with marks ≥ 40 will appear.

---

7. Creating a View with ORDER BY

Depending on the database system and query context, `ORDER BY` can be used in a view definition, but you should **not depend on the view itself guaranteeing output order**.

Better practice:

```
CREATE VIEW student_marks AS
SELECT sid, name, marks
FROM student;
```

Then:

```
SELECT *
FROM student_marks
ORDER BY marks DESC;
```

Important

> A view represents a result set, but the final ordering should normally be specified in the query that reads the view.

---

8. View with Calculated Columns

Suppose we have:

```
student
```

with:

```
sid
name
marks
```

We can create:

```
CREATE VIEW student_result AS
SELECT
    sid,
    name,
    marks,
    marks + 5 AS bonus_marks
FROM student;
```

Now:

```
SELECT *
FROM student_result;
```

Output:

```
sid   name     marks   bonus_marks
---------------------------------
101   Rahul     80         85
102   Aman      70         75
```

---

9. View with Aliases

We can give columns meaningful names.

```
CREATE VIEW student_info AS
SELECT
    sid AS student_id,
    name AS student_name,
    marks AS student_marks
FROM student;
```

Now:

```
SELECT *
FROM student_info;
```

---

10. View Using Aggregate Functions

Views can be created using aggregate functions.

Suppose:

```
student
```

contains:

```
sid   name      course
---------------------
1     Rahul     Java
2     Aman      Java
```

```
3     Priya     Python
4     Neha      Java
5     Raj       Python
```

Create:

```
CREATE VIEW course_student_count AS
SELECT
    course,
    COUNT(*) AS total_students
FROM student
GROUP BY course;
```

Now:

```
SELECT *
FROM course_student_count;
```

Output:

```
course     total_students
------------------------
Java            3
Python          2
```

---

11. View Using JOIN

This is one of the **most important uses of views**.

Suppose we have two tables.

Student

```
CREATE TABLE student (
    sid INT,
    name VARCHAR(50),
    cid INT
);
```

Course

```
CREATE TABLE course (
    cid INT,
    course_name VARCHAR(50),
    fees INT
);
```

Normally we write:

```
SELECT
    s.sid,
    s.name,
    c.course_name,
    c.fees
FROM student s
JOIN course c
ON s.cid = c.cid;
```

If this query is used frequently, create a view:

```
CREATE VIEW student_course_details AS
SELECT
    s.sid,
    s.name,
    c.course_name,
```

```
    c.fees
FROM student s
JOIN course c
ON s.cid = c.cid;
```

Now:

```
SELECT *
FROM student_course_details;
```

Much easier!

---

12. View with JOIN + WHERE

```
CREATE VIEW expensive_course_students AS
SELECT
    s.sid,
    s.name,
    c.course_name,
    c.fees
FROM student s
JOIN course c
ON s.cid = c.cid
WHERE c.fees > 50000;
```

Query:

```
SELECT *
FROM expensive_course_students;
```

---

13. View with JOIN + GROUP BY

Suppose we want the number of students in every course.

```
CREATE VIEW course_summary AS
SELECT
    c.course_name,
    COUNT(s.sid) AS student_count
FROM course c
LEFT JOIN student s
ON c.cid = s.cid
GROUP BY c.cid, c.course_name;
```

Now:

```
SELECT *
FROM course_summary;
```

Possible result:

```
course_name      student_count
------------------------------
Java                     25
Python                   18
MERN                     12
Data Science              9
```

---

14. Querying a View

A view can be queried almost like a table.

```
SELECT *
FROM student_details;
```

WHERE

```
SELECT *
FROM student_details
WHERE marks > 80;
```

ORDER BY

```
SELECT *
FROM student_details
ORDER BY marks DESC;
```

GROUP BY

```
SELECT course, COUNT(*)
FROM student_details
GROUP BY course;
```

JOIN

A view can also be joined with tables.

```
SELECT v.name, c.course_name
FROM student_details v
JOIN course c
ON v.cid = c.cid;
```

---

15. Creating a View with SELECT *

You can technically write:

```
CREATE VIEW student_view AS
SELECT *
FROM student;
```

But this is **not always recommended**.

Why?

Suppose later you modify the table:

```
ALTER TABLE student
ADD phone VARCHAR(15);
```

Your view's behavior may depend on the database system and how the view metadata was stored.

Better Practice

Explicitly specify columns:

```
CREATE VIEW student_view AS
SELECT sid, name, email, course
FROM student;
```

This makes the view easier to understand and maintain.

---

16. Replacing a View

In MySQL, you can use:

```
CREATE OR REPLACE VIEW student_details AS
SELECT sid, name, course
FROM student
WHERE marks >= 50;
```

This modifies the existing view if it exists.

---

17. ALTER VIEW

In MySQL, you can also use:

```
ALTER VIEW student_details AS
SELECT sid, name, course, marks
FROM student;
```

The definition of the view is changed.

---

18. DROP VIEW

If you don't need a view anymore:

```
DROP VIEW student_details;
```

Multiple views:

```
DROP VIEW view1, view2;
```

Important

Dropping a view **does not delete the underlying table data**.

For example:

```
student table
     ↓
student_view
```

If you execute:

```
DROP VIEW student_view;
```

The `student` table remains.

---

19. View Does Not Normally Store Data

This is one of the most important concepts.

Suppose:

```
student
```

contains:

```
101 Rahul 85
```

View:

```
CREATE VIEW student_view AS
SELECT sid, name, marks
FROM student;
```

If we update the original table:

```
UPDATE student
SET marks = 95
WHERE sid = 101;
```

Then:

```
SELECT *
FROM student_view;
```

will normally show:

```
101 Rahul 95
```

Why?

Because the view is based on the underlying table data.

---

20. Advantages of Views

1. Simplicity

Complex SQL queries can be hidden behind a simple view.

Instead of:

```
SELECT ...
FROM student
JOIN course ...
JOIN fees ...
WHERE ...
GROUP BY ...
```

We can simply use:

```
SELECT *
FROM student_report;
```

---

2. Security

Views can restrict access to sensitive columns.

Suppose:

```
employee
```

contains:

```
emp_id
name
salary
email
```

```
password
phone
```

We don't want every user to see salary or password.

Create:

```
CREATE VIEW employee_public AS
SELECT emp_id, name, email, phone
FROM employee;
```

Users can access:

```
SELECT *
FROM employee_public;
```

without directly exposing sensitive columns.

> Note: A view is only part of a security design. Proper privileges/roles are also required.

---

3. Reusability

A frequently used query can be saved as a view.

---

4. Abstraction

Users don't need to know the complexity of the underlying database.

For example:

```
Underlying database

student
course
payment
attendance
     ↓
     ↓
student_report
     ↓
User
```

The user only sees:

```
SELECT *
FROM student_report;
```

---

5. Consistency

If everyone uses the same view, the same business logic can be reused.

For example:

```
CREATE VIEW passed_students AS
SELECT *
FROM student
WHERE marks >= 40;
```

Everyone uses:

```
SELECT *
```

```
FROM passed_students;
```

instead of writing different conditions.

---

21. Disadvantages of Views

Views also have limitations.

1. Complex views can be slow

If a view contains:

- multiple joins
- subqueries
- aggregate functions
- complex calculations

the query may become expensive.

---

2. Not every view is updatable

Some views can be modified directly, while others cannot.

For example, views involving:

```
GROUP BY
DISTINCT
aggregate functions
UNION
certain joins
subqueries
```

may not be directly updatable.

---

3. Dependency

A view depends on its underlying tables.

If the underlying structure changes significantly, the view may need modification.

---

4. Too many views can make databases difficult to manage

Creating hundreds of unnecessary views can make database architecture confusing.

---

22. Updatable Views

An **updatable view** is a view through which we can perform operations such as:

```
INSERT
UPDATE
DELETE
```

under the restrictions of the database system and the view definition.

Example:

```
CREATE VIEW student_basic AS
SELECT sid, name, marks
FROM student;
```

We may be able to update through the view:

```
UPDATE student_basic
SET marks = 90
WHERE sid = 101;
```

The underlying table:

```
student
```

is then updated.

---

23. Updating Through a View

Example:

```
CREATE VIEW student_view AS
SELECT sid, name, marks
FROM student
WHERE marks >= 40;
```

Then:

```
UPDATE student_view
SET marks = 85
WHERE sid = 101;
```

If the view is updatable, the corresponding row in `student` is updated.

Important Concept

```
UPDATE VIEW
     ↓
Underlying Table
     ↓
Data changes
     ↓
View reflects new data
```

---

24. Why Some Views Are Not Updatable

Consider:

```
CREATE VIEW course_count AS
SELECT course, COUNT(*) AS total
FROM student
GROUP BY course;
```

Can we do:

```
UPDATE course_count
SET total = 100
WHERE course = 'Java';
```

Generally, this doesn't make sense.

Why?

Because:

```
total = COUNT(*)
```

is a **calculated aggregate value**, not a stored column that corresponds to one particular row.

---

25. Common Non-Updatable View Situations

A view may not be directly updatable when it contains things such as:

Aggregate functions

```
COUNT()
SUM()
AVG()
MIN()
MAX()
```

GROUP BY

```
GROUP BY course
```

DISTINCT

```
SELECT DISTINCT course
```

UNION

```
SELECT ...
UNION
SELECT ...
```

Certain complex joins

Certain subqueries

The exact rules depend on the database system.

---

26. WITH CHECK OPTION

This is an **important interview and practical concept**.

Suppose:

```
CREATE VIEW passed_students AS
SELECT sid, name, marks
FROM student
WHERE marks >= 40;
```

The view only displays students having:

```
marks >= 40
```

But if the view is updatable, what happens if someone tries:

```
UPDATE passed_students
SET marks = 20
WHERE sid = 101;
```

The row would no longer satisfy:

```
marks >= 40
```

To prevent this, use:

```
CREATE VIEW passed_students AS
SELECT sid, name, marks
FROM student
WHERE marks >= 40
WITH CHECK OPTION;
```

Now the database prevents modifications through the view that would cause the row to fall outside the view's condition.

---

27. Understanding WITH CHECK OPTION

Without:

```
WITH CHECK OPTION
```

the view condition is primarily used for what rows the view displays.

With:

```
WITH CHECK OPTION
```

the condition must also remain true for modifications made through the view.

Remember

> **WITH CHECK OPTION ensures that inserted/updated rows through the view satisfy the view's WHERE condition.**

---

28. Example of WITH CHECK OPTION

Create table:

```
CREATE TABLE student (
    sid INT,
    name VARCHAR(50),
    marks INT
);
```

Create view:

```
CREATE VIEW passed_students AS
SELECT sid, name, marks
FROM student
WHERE marks >= 40
WITH CHECK OPTION;
```

Now:

```
UPDATE passed_students
SET marks = 80
WHERE sid = 101;
```

Allowed.

But:

```
UPDATE passed_students
SET marks = 20
WHERE sid = 101;
```

Not allowed through the view because:

```
20 < 40
```

---

29. View Based on Multiple Tables

Views can combine information from multiple tables.

Example:

Student

```
sid
name
cid
```

Course

```
cid
course_name
fees
```

View

```
CREATE VIEW student_course AS
SELECT
    s.sid,
    s.name,
    c.course_name,
    c.fees
FROM student s
INNER JOIN course c
ON s.cid = c.cid;
```

Now students and courses appear together.

---

30. View vs Subquery

These are commonly confused.

Subquery

```
SELECT *
FROM (
    SELECT sid, name, marks
    FROM student
    WHERE marks > 80
) AS temp;
```

The query is used **only within that statement**.

View

```
CREATE VIEW toppers AS
SELECT sid, name, marks
FROM student
WHERE marks > 80;
```

Now it can be reused:

```
SELECT *
FROM toppers;
```

Main Difference

Subquery	View
Usually temporary query component	Named database object
Created inside a query	Created using `CREATE VIEW`
Not normally reusable by name	Reusable
Exists as part of query	Exists until dropped
Good for one-time logic	Good for repeated logic

---

31. View vs Temporary Table

Another common interview question.

View	Temporary Table
Virtual/logical table	Actual temporary table
Usually doesn't store result data separately	Stores rows temporarily
Definition persists until dropped	Usually exists for session/transaction depending on DB
Data comes from underlying tables	Data is physically/materially held temporarily
Good for abstraction	Good for intermediate processing

Example temporary table:

```
CREATE TEMPORARY TABLE temp_student AS
SELECT *
FROM student;
```

---

32. View vs Materialized View

This is an advanced concept.

Normal View

```
View
 ↓
Stores query definition
 ↓
Reads underlying tables when queried
```

Materialized View

```
Materialized View
 ↓
Stores the query result
```

```
  ↓
Must be refreshed when underlying data changes
```

Materialized views are particularly useful for expensive reporting/aggregation queries, but support varies by database system.

Important

MySQL traditionally does not provide native materialized views like Oracle/PostgreSQL extensions do. In MySQL, similar behavior can be implemented using tables plus refresh logic.

---

33. Real-World Example

Imagine an e-commerce database.

Tables:

```
customers
orders
products
payments
```

A manager wants:

```
Customer Name
Order ID
Product
Amount
Payment Status
```

Instead of repeatedly writing a large query:

```
SELECT
    c.name,
    o.order_id,
    p.product_name,
    o.amount,
    pay.status
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN products p
    ON o.product_id = p.product_id
JOIN payments pay
    ON o.order_id = pay.order_id;
```

Create:

```
CREATE VIEW order_report AS
SELECT
    c.name,
    o.order_id,
    p.product_name,
    o.amount,
    pay.status
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN products p
    ON o.product_id = p.product_id
JOIN payments pay
    ON o.order_id = pay.order_id;
```

Now manager can simply use:

```
SELECT *
FROM order_report;
```

---

34. Views for Security

Suppose an employee table contains:

```
employee_id
name
email
phone
salary
bank_account
password
```

We can expose only safe columns:

```
CREATE VIEW employee_directory AS
SELECT
    employee_id,
    name,
    email,
    phone
FROM employee;
```

Now the application/reporting user can be granted access to the view rather than the underlying sensitive table.

This is a very common use of views.

---

35. Granting Permission on a View

In systems such as MySQL, privileges can be given on a view.

Conceptually:

```
GRANT SELECT
ON database_name.employee_directory
TO 'user'@'localhost';
```

The exact privilege setup depends on your database configuration.

This allows a user to query:

```
SELECT *
FROM employee_directory;
```

without necessarily giving them unrestricted access to the underlying table.

---

36. View Naming Convention

Good names:

```
student_details
student_report
passed_students
course_summary
employee_directory
order_report
monthly_sales
```

Avoid confusing names:

```
view1
abc
test
newview
xyz
```

A view name should describe what the view represents.

---

37. Complete Example for Students

Let's create a small database.

Step 1 — Student table

```
CREATE TABLE student (
    sid INT PRIMARY KEY,
    name VARCHAR(50),
    course VARCHAR(50),
    marks INT,
    city VARCHAR(50)
);
```

Step 2 — Insert data

```
INSERT INTO student
VALUES
(1, 'Rahul', 'Java', 85, 'Delhi'),
(2, 'Aman', 'Python', 72, 'Gurgaon'),
(3, 'Priya', 'Java', 91, 'Delhi'),
(4, 'Neha', 'MERN', 65, 'Noida'),
(5, 'Raj', 'Python', 88, 'Delhi');
```

---

38. Basic View

```
CREATE VIEW student_details AS
SELECT sid, name, course
FROM student;
```

Check:

```
SELECT *
FROM student_details;
```

---

39. Toppers View

```
CREATE VIEW toppers AS
SELECT sid, name, course, marks
FROM student
WHERE marks >= 80;
```

Query:

```
SELECT *
FROM toppers;
```

---

40. Delhi Students View

```
CREATE VIEW delhi_students AS
SELECT sid, name, course, marks
FROM student
WHERE city = 'Delhi';
```

Query:

```
SELECT *
FROM delhi_students;
```

---

41. Java Students View

```
CREATE VIEW java_students AS
SELECT sid, name, marks
FROM student
WHERE course = 'Java';
```

---

42. Aggregate View

```
CREATE VIEW course_summary AS
SELECT
    course,
    COUNT(*) AS total_students,
    AVG(marks) AS average_marks,
    MAX(marks) AS highest_marks,
    MIN(marks) AS lowest_marks
FROM student
GROUP BY course;
```

Now:

```
SELECT *
FROM course_summary;
```

---

43. Nested Use of a View

A view can be used in another query.

```
SELECT *
FROM course_summary
WHERE average_marks > 75;
```

This is useful for reporting.

---

44. Checking Existing Views in MySQL

To see tables:

```
SHOW TABLES;
```

Depending on the database/client, views may also appear among the table-like objects.

To inspect a view:

```
SHOW CREATE VIEW student_details;
```

This shows the SQL used to create the view.

---

45. Dropping a View Safely

```
DROP VIEW IF EXISTS student_details;
```

This avoids an error if the view doesn't exist.

---

46. Important Commands — Quick Revisior

Create

```
CREATE VIEW view_name AS
SELECT ...
```

Create or replace

```
CREATE OR REPLACE VIEW view_name AS
SELECT ...
```

Alter

```
ALTER VIEW view_name AS
SELECT ...
```

Read

```
SELECT *
FROM view_name;
```

Drop

```
DROP VIEW view_name;
```

Safe drop

```
DROP VIEW IF EXISTS view_name;
```

Check definition in MySQL

```
SHOW CREATE VIEW view_name;
```

---

47. Most Important Interview Questions

Q1. What is a view?

A view is a virtual table based on the result of a SQL query.

Q2. Does a normal view store data?

Generally, no. It stores the query definition and retrieves data from its underlying objects when queried.

Q3. Can we query a view?

Yes.

```
SELECT *
FROM view_name;
```

Q4. Can we create a view using JOIN?

Yes.

```
CREATE VIEW student_course AS
SELECT ...
FROM student
JOIN course
ON ...;
```

Q5. Can a view contain aggregate functions?

Yes.

```
COUNT()
SUM()
AVG()
MIN()
MAX()
```

However, such views are generally not directly updatable.

Q6. Can we update a view?

Some views are updatable, depending on their definition and database rules.

Q7. Can we delete a view?

Yes:

```
DROP VIEW view_name;
```

Q8. Does dropping a view delete the original table?

No.

```
DROP VIEW
    ↓
Only view is removed
    ↓
Base table remains
```

Q9. What is WITH CHECK OPTION?

It ensures that rows modified through an updatable view continue to satisfy the view's filtering condition.

Q10. What is the difference between a view and a table?

A table normally stores actual rows, while a normal view represents the result of a query over underlying tables/views.

---

48. Common Student Mistakes

Mistake 1

Writing:

```
CREATE VIEW student_view
SELECT *
FROM student;
```

✕ Missing `AS`.

Correct:

```
CREATE VIEW student_view AS
SELECT *
FROM student;
```

Mistake 2

Thinking a view is a completely separate copy of the table.

✕ Incorrect.

A normal view is a logical/virtual representation based on a query.

Mistake 3

Thinking:

```
DROP VIEW student_view;
```

deletes:

```
student
```

✕ Incorrect.

Only the view is removed.

Mistake 4

Assuming every view is automatically updatable.

✕ Incorrect.

Complex views may not be updatable.

Mistake 5

Putting `ORDER BY` inside a view and assuming every later query will return rows in that order.

Better:

```
SELECT *
FROM my_view
ORDER BY marks DESC;
```

---

49. Practice Questions for Students

Use this table:

```
CREATE TABLE student (
    sid INT PRIMARY KEY,
    name VARCHAR(50),
    course VARCHAR(50),
    marks INT,
    city VARCHAR(50),
    fees INT
);
```

Beginner Level

Q1. Create a view showing:

```
sid
name
course
```

Q2. Create a view containing only students who scored 80 or more.

Q3. Create a view containing only Delhi students.

Q4. Display all records from a view.

Q5. Display only names from a view.

Q6. Sort a view by marks descending.

Q7. Drop a view.

Intermediate Level

Q8. Create a view showing:

```
course
total students
```

Q9. Create a view showing:

```
course
average marks
highest marks
lowest marks
```

Q10. Create a view containing students whose fees are greater than 50,000.

Q11. Create a view showing only Java students.

Q12. Create a view showing students whose marks are between 60 and 90.

Q13. Create a view with calculated grade.

For example:

```
marks >= 80 → A
marks >= 60 → B
marks >= 40 → C
otherwise → F
```

Advanced Level

Q14. Create a view using two tables:

```
student
course
```

Q15. Create a view showing every course and the number of students enrolled.

Q16. Create a view showing:

```
course
average marks
```

and then display courses whose average marks are greater than 75.

Q17. Create a view with `WITH CHECK OPTION`.

Q18. Determine whether a particular view is updatable.

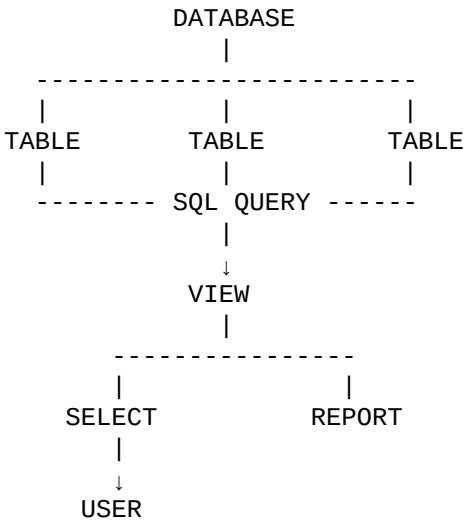
Q19. Create an employee view that hides salary.

Q20. Create a sales-report view using:

```
customer
orders
product
payment
```

---

50. Final Revision Diagram



Remember These 7 Points

1. **View = Virtual/Logical Table**
2. Created using **`CREATE VIEW`**